



OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Design Concepts

Archana A

Department of Computer Applications

Design concepts provide the **necessary framework**
“to get the thing on right way”.

1. Abstraction.
2. Architecture.
3. Patterns.
4. Separation of concerns.
5. Modularity
6. Information Hiding.
7. Functional independence.
8. Refinement
9. Aspects
10. Refactoring.
11. Object Oriented design concepts
12. Design Classes.

- An abstraction is a tool that enables a designer **to consider a component at an abstract level - without bothering about the internal details** of the implementation.
- Abstraction can be used for existing element as well as the component being designed.
- For modular solution there are many level of abstraction
 1. **Higher level abstraction**
 2. **Lower level abstraction**
 3. **Procedural abstraction**
 4. **Data abstraction**

1. At the **highest level of abstraction** – a solution is stated in **broad terms**
 - High-level abstraction deals with the broader, **more generalized aspects** of a system or concept.
 - It focuses on the **overall structure, functionality, and interactions** within the system.
 - High-level abstractions often **hide implementation details** and focus on the **system's external behavior and interfaces**.
 - **Examples** of high-level abstractions include **system architecture diagrams, flowcharts, and conceptual models**

2. At **lower level of abstraction** – a more detailed description of the solution is provided.

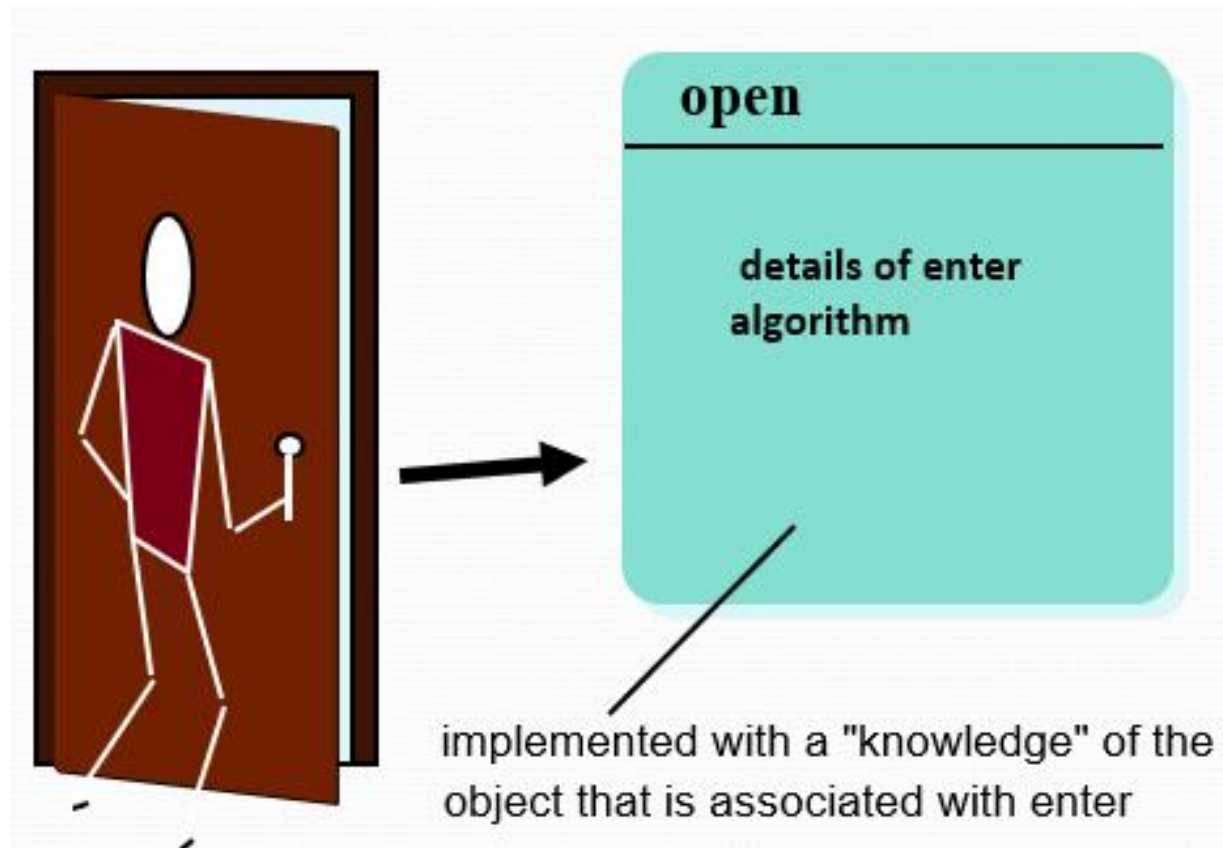
- Low-level abstraction involves **detailed, specific representations of components or elements within a system.**
- It deals with the quality of complex implementation, **including hardware components, algorithms, and data structures.**
- Low-level abstractions **expose the inner workings of a system**, including how individual components interact and are implemented.
- Examples of low-level abstractions include **source code, circuit diagrams, assembly language, and detailed algorithms.**
- Engineers use low-level abstractions to delve into specific implementation details, optimize performance, and debug issues at a granular level.

3. Procedural Abstraction: Sequence of instructions that have a **specific and limited** function.

Ex. ***Open*** a door

- ***open*** implies long sequence of activities i.e., **walk to the door, grasp knob, turn knob and pull the door**, etc.
- In Procedural / Functional abstraction
 - (i) A module is specified by the method it performs.
 - (ii) The details of the algorithm to accomplish the functions are not visible to the user of the function.
- Functional abstraction forms the basis for **Function oriented design approaches**.

Procedural Abstraction:



4. Data Abstraction: Collection of data that describes a data object.

Ex. Open a door. – *door* is data object.

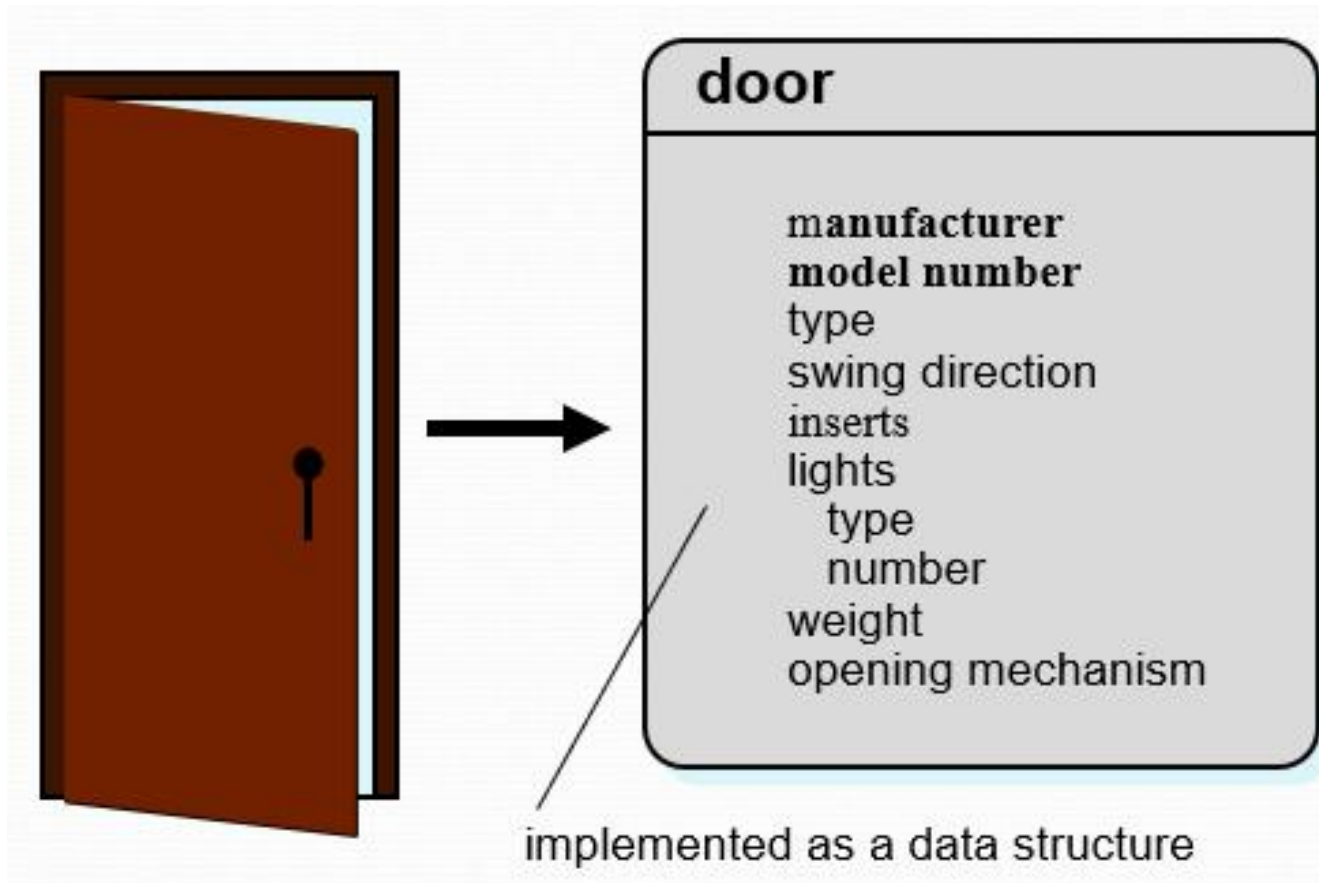
- Data abstraction for *door* would encompass a set of attributes that describe the door.

Ex: door type, swing direction, opening mechanism, etc.

- Details of the data elements are not visible to the users of data. Data Abstraction forms the basis for **Object Oriented design approaches**.

.

Data Abstraction:



- Software architecture is the **structure** or **organization** of program **components** (**modules**), the manner in which these **components** **interact**, and the **structure of data that are used** by these components.

Software architecture suggest “ **the overall structure of the software** and the ways in which that structure provides **conceptual integrity for a system.**”

There are **different models**, which can be used to represent architecture.

- 1. Structural Model**- represent architecture as an *organized collection of program components* . Example: **Class diagram, object diagram** etc.
- 2. Framework model** – Increase level of design abstraction by identifying *repeatable architectural design framework*.
for example: **microservices architecture**

3. Dynamic model – addresses the behavior aspects of the program architecture and indicating *how structure or system configuration* may change as a function of **external event**.

4. Process Model – focus on **design of the business** or **technical process** that the system must accommodate.

5. Functional models – used to represent the *functional hierarchy* of a system.

These five models are developed by using Architecture Description Language (ADL).

- A design pattern describes a **design structure** that **solves** a particular **design problem** within a specific **context**.
- The **intent of each design pattern** is to provide a description that **enables a designer to determine**
 - 1) whether the **pattern is applicable** to the current work,
 - 2) whether the pattern can be **reused** (hence, saving design time), and
 - 3) whether the pattern can **serve as a guide** for developing a similar, but functionally or structurally different pattern.



THANK YOU

Ms. Archana A

Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392